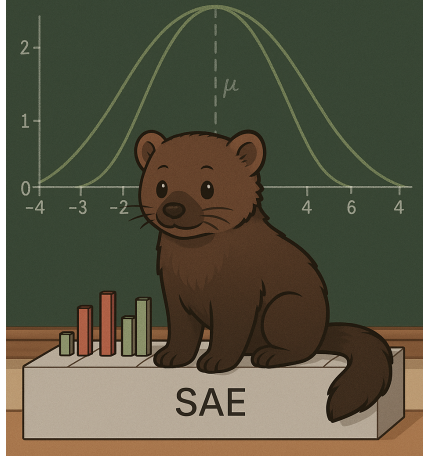

Fisher Information-Guided Sparsity in Autoencoders

 **Rui Melo**
FEUP FACULDADE DE ENGENHARIA
UNIVERSIDADE DE PORTO



Abstract

Sparse Autoencoders (SAEs) have emerged as powerful tools for extracting interpretable, semantically meaningful features from the internal activations of large language models (LLMs). However, existing SAE training schemes often suffer from co-activation and redundancy, limiting their effectiveness in mechanistic interpretability. We introduce **Fisher SAEs**, a principled extension of different SAE architectures based on the addition of a Fisher regularisation Term, a Fisher Information-based regularisation term to penalise feature co-activation. This regulariser encourages disentangled, decorrelated representations while preserving reconstruction fidelity. We evaluate our approach on code understanding tasks, applying it across multiple activation types, including residual streams, MLP outputs, and attention outputs, at various transformer layers. Empirical results show a Pareto improvement by improving sparsity and representation quality over baseline methods, without sacrificing fidelity, and enabling more interpretable feature decompositions. Our approach advances the utility of SAEs in LLM interpretability by promoting both selective and orthogonal latent structures.

1 Introduction

Transformer-based Large Language Models (LLMs) [19] have catalysed significant interest in *mechanistic interpretability*, a subfield focused on understanding internal model computations through modular, human-interpretable components known as *circuits* [12, 11]. These circuits, which can be viewed as subgraphs of a model’s computational graph responsible for specific behaviours, provide a structured lens through which to analyse complex model functionality. Concurrently, Sparse Autoencoders (SAEs) have emerged as promising tools for recovering semantically meaningful latent features from neural activations [4, 15, 1].

SAEs enable identification of causally relevant and often interpretable directions within the activation space of Language Models (LMs) [1, 4, 18]. This has sparked growing interest in using sparse decompositions from SAEs for downstream tasks such as circuit analysis [9], classification [10], model editing or steering [3].

The core idea behind SAEs is to approximate model activations using sparse linear combinations from a large dictionary of basic feature directions. Two central desiderata for such decompositions are *sparsity*, the requirement that only a few dictionary elements are active per input, and *faithfulness*, the requirement that reconstructed activations closely match the original in a meaningful metric [1]. These goals are inherently at odds: for any fixed dictionary and training regime, increasing sparsity generally comes at the cost of reduced reconstruction fidelity.

Recent work has focused on pushing the Pareto frontier, balancing sparsity and reconstruction quality through architectural innovations and new training schemes [13, 7, 17]. A unifying element across these advances is the introduction of thresholding or gating mechanisms to control which SAE features are included in each decomposition.

Despite these improvements, including methods such as JumpReLU and gated autoencoders, feature redundancy remains a challenge. In practice, the degree of sparsity achieved by current methods may still be insufficient for some interpretability tasks.

In this work, we propose **Fisher SAEs**, a principled extension of different SAE architectures [14, 5]. Our approach introduces a Fisher [6, 2] Information-based regularisation term that penalises co-activations across features, encouraging decorrelated and more interpretable basis vectors. Empirical results demonstrate that Fisher SAEs improves decomposition quality and downstream interpretability without compromising reconstruction fidelity. This method advances the utility of SAEs in mechanistic interpretability by fostering sparser and more semantically distinct representations.

2 Preliminaries

In this section, similar to [5], we outline the fundamental concepts and architectures relevant to SAEs, especially those incorporating the prior JumpReLU activation and our Fisher SAEs term. Our aim is to provide a foundation for the methods proposed in this paper, building on prior formulations of encoder-decoder-based sparse decomposition.

Sparse Autoencoder Structure SAEs are models designed to extract interpretable, sparse representations of input activations $\mathbf{x} \in \mathbb{R}^n$. These representations are computed via a learned overcomplete basis (or *dictionary*) of feature vectors, where the number of features M greatly exceeds the input dimension n . Encoding and decoding are defined as follows:

$$\begin{aligned}\mathbf{f}(\mathbf{x}) &= \sigma(\mathbf{W}_{\text{enc}}\mathbf{x} + \mathbf{b}_{\text{enc}}), \\ \hat{\mathbf{x}}(\mathbf{f}) &= \mathbf{W}_{\text{dec}}\mathbf{f} + \mathbf{b}_{\text{dec}}\end{aligned}$$

Here, $\mathbf{f}(\mathbf{x}) \in \mathbb{R}^M$ denotes the sparse, non-negative feature vector, and $\hat{\mathbf{x}}(\mathbf{f})$ is the reconstruction of the input. The matrix $\mathbf{W}_{\text{dec}}$ consists of column vectors $\mathbf{d}_i$ forming the learned dictionary.

Nonlinear Activation Functions The nonlinearity σ is crucial in enforcing sparsity and varies across architectures. Common choices include ReLU, TopK [7], and JumpReLU [14]. While ReLU applies an element-wise threshold at zero, TopK retains only the K largest pre-activations. JumpReLU introduces a trainable threshold $\theta > 0$:

$$\text{JumpReLU}_{\theta}(z) := z \cdot H(z - \theta), \quad (1)$$

where H is the Heaviside step function. This function retains values only if they exceed the threshold, effectively gating activation.

Training Objective Training is typically carried out by minimising a reconstruction loss combined with a sparsity-inducing penalty:

$$\mathcal{L}(\mathbf{x}) = \underbrace{\|\mathbf{x} - \hat{\mathbf{x}}(\mathbf{f}(\mathbf{x}))\|_2^2}_{\text{Reconstruction loss}} + \underbrace{\lambda \cdot S(\mathbf{f}(\mathbf{x}))}_{\text{Sparsity term}} + \mathcal{L}_{\text{aux}}, \quad (2)$$

improve this.
We are no
longer tied to
JumpReLU

where λ is a regularization coefficient and $S(\cdot)$ is typically an L1 norm. The auxiliary term $\mathcal{L}_{\text{aux}}$ may encourage gradient flow for parameters with weak supervision or help maintain feature diversity. In this paper, our $\mathcal{L}_{\text{aux}}$ is our Fisher Regularisation Term.

Sparsity Regularizers To encourage interpretability and compression, various forms of sparsity constraints have been studied. The L1 norm $\|\mathbf{f}\|_1$ is common, but it has the downside of penalising magnitude as well as sparsity and is sensitive to reparameterization. An improved version, the RI-L1 penalty, is defined as:

$$S_{\text{RI-L1}}(\mathbf{f}) = \sum_{i=1}^M f_i \|\mathbf{d}_i\|_2, \quad (3)$$

which remains invariant to rescalings of encoder and decoder weights.

JumpReLU-Based Architectures In JumpReLU SAEs, the encoder becomes:

$$\mathbf{f}(\mathbf{x}) = \text{JumpReLU}_{\boldsymbol{\theta}}(\mathbf{W}_{\text{enc}}\mathbf{x} + \mathbf{b}_{\text{enc}}), \quad (4)$$

where $\boldsymbol{\theta} \in \mathbb{R}_+^M$ denotes per-feature thresholds. These thresholds decouple activation detection from magnitude estimation, akin to mechanisms in TopK and Gated SAEs. The associated loss uses an L0-like penalty:

$$\mathcal{L}(\mathbf{x}) = \|\mathbf{x} - \hat{\mathbf{x}}(\mathbf{f}(\mathbf{x}))\|_2^2 + \lambda \cdot \|\mathbf{f}(\mathbf{x})\|_0. \quad (5)$$

This formulation better isolates sparse structure but complicates gradient computation due to the non-differentiability of the L0 norm. To address this, we apply straight-through estimators (STEs) to provide surrogate gradients.

3 Fisher SAEs

Fisher SAEs is a targeted modification of the SAE architecture, which includes a new loss term based on the Fisher Matrix [6, 2]. Maintaining the original activation functions to zero out pre-activations below a predetermined positive threshold, in the case of JumpReLU, we now focus on reducing the redundancy of the activated features.

To enhance the quality of learned representations, we introduce a novel regularisation term, **Fisher regularisation Term**, into the training objective. This term discourages co-activation among features by estimating an empirical Fisher Information Matrix over the pre-activations of the encoder.

To estimate feature sensitivity in a meaningful local neighbourhood of the representation space, we define a *pseudo-loss* over the perturbed pre-activations as the squared ℓ_2 norm:

$$\mathcal{L}_{\text{pseudo}} = \sum_i \tilde{h}_i^2. \quad (6)$$

This serves as a proxy for the network’s local sensitivity to latent activation changes.

We compute the gradient of this pseudo-loss with respect to the feature activations (after the non-linear activation passing):

$$\mathbf{g} = \nabla_{\mathbf{h}} \mathcal{L}_{\text{pseudo}}. \quad (7)$$

These gradients serve as an empirical estimate of local sensitivity, analogous to directions of high curvature in the loss landscape.

Using these gradients, we construct the empirical Fisher Information Matrix by averaging the outer product across a batch of size B :

$$F_{\text{empirical}} = \frac{1}{B} \sum_{b=1}^B \mathbf{g}_b \mathbf{g}_b^\top, \quad (8)$$

where $\mathbf{g}_b \in \mathbb{R}^d$ is the gradient vector for the b -th sample. This matrix captures pairwise sensitivity relationships between latent features. Mathematically, $F_{\text{empirical}}$ corresponds to the *second moment matrix* (i.e., the uncentered covariance) of the gradient vectors:

$$F_{\text{empirical}} = \mathbb{E}[\mathbf{g}\mathbf{g}^\top].$$

While this differs from the true covariance matrix (which subtracts the mean), the uncentered version is sufficient for revealing correlations in activation sensitivity. Penalising these correlations helps the network disentangle redundant or co-dependent features.

To enforce independence among the learned features, we penalise the off-diagonal entries of $F_{\text{empirical}}$:

$$\mathcal{L}_{\text{Fisher}} = \theta \sum_{i \neq j} \left(F_{\text{empirical}}^{(i,j)} \right)^2, \quad (9)$$

where θ controls the strength of the regularisation. This encourages the model to learn decorrelated, functionally distinct features in the latent space.

The complete training objective combines reconstruction accuracy, sparsity, and Fisher-based decorrelation:

$$\mathcal{L}_{\text{total}}(\mathbf{x}) = \underbrace{\|\mathbf{x} - \hat{\mathbf{x}}(\mathbf{f}(\mathbf{x}))\|_2^2}_{\mathcal{L}_{\text{reconstruction}}} + \underbrace{\lambda \cdot \|\mathbf{f}(\mathbf{x})\|_0}_{\mathcal{L}_{\text{sparsity}}} + \underbrace{\theta \sum_{i \neq j} \left(F_{\text{empirical}}^{(i,j)} \right)^2}_{\mathcal{L}_{\text{Fisher}}}, \quad (10)$$

where λ and θ control the sparsity and decorrelation regularization strengths, respectively.

We refer to this modified model as Fisher SAEs. Empirical results demonstrate that Fisher SAEs achieves improved decomposition quality without sacrificing reconstruction fidelity, making it a promising approach for interpretable representation learning.

4 Methodology

Our primary objective is to improve the training of SAEs for code representation learning. In particular, we focus on enhancing sparsity and interpretability without degrading reconstruction fidelity. To this end, we propose Fisher SAEs, which augments the JumpReLU framework with a Fisher Information-based regularisation term that penalises co-activation among latent features.

The motivation for this work is visualised in Figure 1, which displays the activation patterns of SAE features on the HumanEval dataset [8, 16]. The widespread co-activation of multiple features suggests redundancy and entanglement, underscoring the need for better feature decorrelation.

4.1 Data

We evaluate our method on the Devign dataset [20], which consists of C functions labelled for the presence or absence of security vulnerabilities. This dataset is widely used in code representation learning tasks and is particularly suitable for assessing the interpretability and effectiveness of sparse decompositions. To further test generalizability across programming languages, we also evaluate on TheStack subsets of Java and Python. Finally, to demonstrate applicability beyond the domain of source code, we include an evaluation on Tiny Stories, a commonly used dataset for training SAEs on natural language.

[cite datasets](#)

4.2 Evaluation Metrics

To assess the effectiveness of our model, we evaluate performance across the following key metrics:

- **ℓ_0 norm:** Measures the number of non-zero activations in the latent vector per input. This metric reflects hard sparsity and often correlates with interpretability and compression.
- **ℓ_1 norm:** Captures the magnitude of activations. This softer sparsity metric is differentiable and affects the generalisation and robustness of the learned representations.
- **Mean Squared Error (MSE):** Quantifies the fidelity of reconstructions by measuring the mean squared difference between input and output. Lower values indicate higher reconstruction accuracy.
- **Fisher Penalty:** Computed as the sum of squared off-diagonal elements of the empirical Fisher Information Matrix. This term encourages decorrelation among features and serves as a proxy for disentanglement.

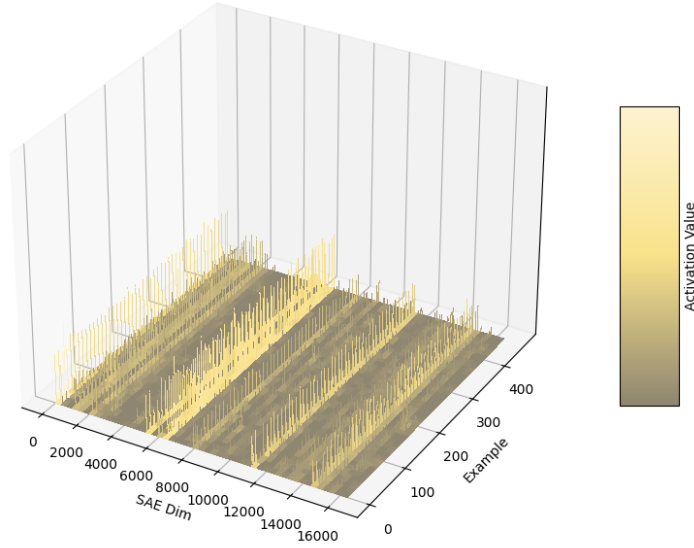


Figure 1: Gemma 2B Layer 17 SAE activations across buggy and safe versions of HumanEval. High co-activation motivates the need for improved sparsity and feature disentanglement [10].

- **L2 ratio:** The normalised ℓ_2 norm of latent activations relative to pre-activations, providing an additional view on representational compactness.

These metrics jointly assess the trade-off between sparsity, fidelity, and feature orthogonality, three critical desiderata for interpretability in SAEs.

4.3 Network Components

To comprehensively assess the generality and robustness of Fisher SAEs, we apply our method across multiple types of internal representations within transformer models. Specifically, we train and evaluate SAEs over the following activation types:

- **Residual Stream:** The core hidden state passed between layers, reflecting the cumulative computation at each point in the model.
- **MLP Output:** The output of the feedforward (MLP) sublayer, capturing non-linear transformations of the residual stream.
- **Attention Output:** The result of the attention mechanism, representing the contextual aggregation of tokens.

TODO

For each of these components, we compare the performance of Fisher SAEs against standard SAEs, JumpReLU-based SAEs and Gated SAEs across several transformer layers. This setup enables us to test the effectiveness of our proposed Fisher-based regularisation in different functional modules and depths of the model, providing insight into where and how Fisher SAEs improves interpretability and sparsity without compromising reconstruction quality. We based our approach on two different models: GPT-2 Small and Gemma 2B. We utilised a standard expansion factor of 16 and our θ term of $1e - 2$.

5 Results

Effect of Fisher Regularisation. As shown in Tables 2 and 3, applying Fisher-based regularisation consistently enhances sparsity across architectures, with other noticeable reductions in both ℓ_0 and ℓ_1 sparsity metrics. This indicates that the Fisher term effectively encourages more compact

Architecture	Component	Metric	Metric Difference (Fisher - Non-Fisher)			
			Devign	TheStack - Dedup Python	TheStack - Java	Tiny Stories
Gated	MLP Out	ℓ_0	-22.46	-18.36	-15.06	-25.80
		ℓ_1	-2.55	-2.55	-2.03	-2.50
		ℓ_2 Ratio	0.00	0.00	0.00	0.00
		MSE	-0.07	0.04	0.18	-0.53
	Residual	ℓ_0	-30.99	-29.51	-27.17	-27.50
		ℓ_1	-3.49	-3.21	-2.83	-2.77
		ℓ_2 Ratio	0.00	0.00	0.00	0.00
		MSE	0.10	-0.04	-0.18	-0.07
JumpReLU	MLP Out	ℓ_0	-461.62	-456.99	-540.95	-602.81
		ℓ_1	-518.66	-986.07	-1198.53	-240.58
		ℓ_2 Ratio	0.00	0.00	0.00	0.00
		MSE	0.13	0.26	0.12	0.24
	Residual	ℓ_0	-2409.40	-2140.97	-2002.30	-1902.54
		ℓ_1	3.74	-44.62	-144.42	-240.58
		ℓ_2 Ratio	-0.01	-0.01	0.00	0.00
		MSE	0.00	0.00	0.05	0.03
Standard	MLP Out	ℓ_0	-448.97	-444.44	-518.12	-579.10
		ℓ_1	-501.19	-1007.11	-1173.56	-1531.92
		ℓ_2 Ratio	0.00	0.00	0.00	0.00
		MSE	0.11	0.16	0.17	0.36
	Residual	ℓ_0	-1605.63	-2128.21	-1984.46	-1883.72
		ℓ_1	-8.94	-41.29	-151.77	-247.49
		ℓ_2 Ratio	0.01	0.00	-0.01	0.00
		MSE	0.01	0.05	0.05	0.05

Table 1: Comparison of Fisher vs. non-Fisher metric differences across datasets. Negative values (green) indicate improvements in sparsity (ℓ_0 , ℓ_1) or reconstruction (MSE), while positive MSE values (red) suggest degradation. Results show Fisher regularisation consistently improves sparsity metrics across all architectures and datasets, with mixed effects on reconstruction quality. The ℓ_2 ratio remains relatively stable in all configurations. New data shows stronger ℓ_0 improvements for the Gated architecture and higher MSE variance in the Standard architecture.

representations by zeroing out a greater proportion of parameters and reducing the overall activity of the model. The decline in ℓ_1 sparsity further suggests a shift toward sharper, more binary sparsity patterns, rather than diffuse suppression.

Importantly, the ℓ_2 **ratio** remains largely stable across all settings, showing that the magnitude of the remaining (active) parameters is preserved. This implies that although Fisher regularisation prunes more aggressively, it retains the expressive capacity of the model by maintaining the strength of the surviving activations.

While a slight increase in **MSE Loss** is observed, indicating marginal degradation in reconstruction fidelity, these results demonstrate that Fisher-based regularisation can achieve significantly improved sparsity with minimal impact on the representational power of the model. Such a trade-off may be highly beneficial in settings where model efficiency and interpretability are prioritised.

References

- [1] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermy, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Aske, et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, page 2, 2023.
- [2] Dan Coe. Fisher matrices and confidence ellipses: a quick-start guide and software. *arXiv preprint arXiv:0906.4123*, 2009.
- [3] Arthur Conmy and Neel Nanda. Activation steering with SAEs. *Alignment Forum*, 2024. Progress Update #1 from the GDM Mech Interp Team.
- [4] Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*, 2023.
- [5] N Benjamin Erichson, Zhewei Yao, and Michael W Mahoney. Jumprelu: A retrofit defense strategy for adversarial attacks. *arXiv preprint arXiv:1904.03750*, 2019.
- [6] Jonathan R Gair, Andrea Antonelli, and Riccardo Barbieri. A fisher matrix for gravitational-wave population inference. *Monthly Notices of the Royal Astronomical Society*, 519(2):2736–2753, December 2022.
- [7] Leo Gao, Tom Dupré la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders, 2024.
- [8] Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. Impact of code language models on automated program repair. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 1430–1442. IEEE, 2023.
- [9] Samuel Marks, Can Rager, Eric J Michaud, Yonatan Belinkov, David Bau, and Aaron Mueller. Sparse feature circuits: Discovering and editing interpretable causal graphs in language models. *arXiv preprint arXiv:2403.19647*, 2024.
- [10] Rui Melo, Claudia Mamede, Andre Catarino, Rui Abreu, and Henrique Lopes Cardoso. Are sparse autoencoders useful for java function bug detection? *arXiv preprint arXiv:2505.10375*, 2025.
- [11] Chris Olah, Nick Cammarata, Ludwig Schubert, Gabriel Goh, Michael Petrov, and Shan Carter. Zoom in: An introduction to circuits. *Distill*, 5(3):e00024–001, 2020.
- [12] Chris Olah, Arvind Satyanarayan, Ian Johnson, Shan Carter, Ludwig Schubert, Katherine Ye, and Alexander Mordvintsev. The building blocks of interpretability. *Distill*, 3(3):e10, 2018.
- [13] Senthoran Rajamanoharan, Arthur Conmy, Lewis Smith, Tom Lieberum, Vikrant Varma, János Kramár, Rohin Shah, and Neel Nanda. Improving dictionary learning with gated sparse autoencoders, 2024.
- [14] Senthoran Rajamanoharan, Tom Lieberum, Nicolas Sonnerat, Arthur Conmy, Vikrant Varma, János Kramár, and Neel Nanda. Jumping ahead: Improving reconstruction fidelity with jumprelu sparse autoencoders, 2024.
- [15] Lee Sharkey, Dan Braun, and Beren Millidge. Taking features out of superposition with sparse autoencoders. In *AI Alignment Forum*, 2022.
- [16] André Silva, Sen Fang, and Martin Monperrus. Repairllama: Efficient representations and fine-tuned adapters for program repair. *arXiv preprint arXiv:2312.15698*, 2023.
- [17] Glen M. Taggart. Prolu: A nonlinearity for sparse autoencoders. *Alignment Forum*, 2024.

- [18] Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, Hoagy Cunningham, Nicholas L Turner, Callum McDougall, Monte MacDiarmid, C. Daniel Freeman, Theodore R. Sumers, Edward Rees, Joshua Batson, Adam Jermy, Shan Carter, Chris Olah, and Tom Henighan. Scaling monosemanticity: Extracting interpretable features from claude 3 sonnet. *Transformer Circuits Thread*, 2024.
- [19] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [20] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks, 2019.

6 Appendix

We evaluate Fisher SAEs on activations from multiple transformer components to assess sparsity, reconstruction fidelity, and the effect of Fisher regularisation. Experiments are conducted on the Devign dataset.

6.1 GPT-2 Small

6.1.1 Devign

MLP Output

Sparsity. Figure 2 shows that Fisher SAEs consistently promotes sparser representations in MLP outputs across layers 0, 5, and 10. We observe substantial reductions in both ℓ_0 and ℓ_1 sparsity compared to standard and JumpReLU baselines, especially under the gated variant. These results suggest that our method encourages more compact and interpretable representations.

Reconstruction Fidelity. Despite stronger regularisation, reconstruction performance remains stable. As shown in Figure 3, our method achieves comparable or improved MSE and Fisher losses across architectures, highlighting its ability to preserve activation fidelity even under sparsity constraints.

Residual Stream Activations

Sparsity. We observe consistent improvements in sparsity within the residual stream (Figure 4). Across all architectures, Fisher SAEs reduces ℓ_0 and ℓ_1 sparsity metrics more effectively than baselines, confirming the method’s generality across activation types.

Reconstruction Fidelity. Figure 5 shows reconstruction losses remain low and stable, mirroring trends observed in the MLP outputs. This demonstrates the robustness of our technique across structurally distinct parts of the model.

Attention Output

TODO

6.1.2 Summary Across Activation Types

Across MLP outputs and residual stream activations, Fisher SAEs consistently yields:

- **Increased sparsity:** Substantially lower ℓ_0 and ℓ_1 norms compared to standard sparse autoencoders.
- **Stable reconstruction:** Comparable or improved MSE and Fisher losses across model variants.
- **Effective disentanglement:** Consistently lower Fisher penalties suggest better latent factor separation.
- **Robust encoding:** L2 Ratio analysis (not shown) confirms maintained encoding capacity.

These findings indicate that Fisher SAEs improves the interpretability and efficiency of learned representations without compromising fidelity or expressive capacity.

6.1.3 Summary

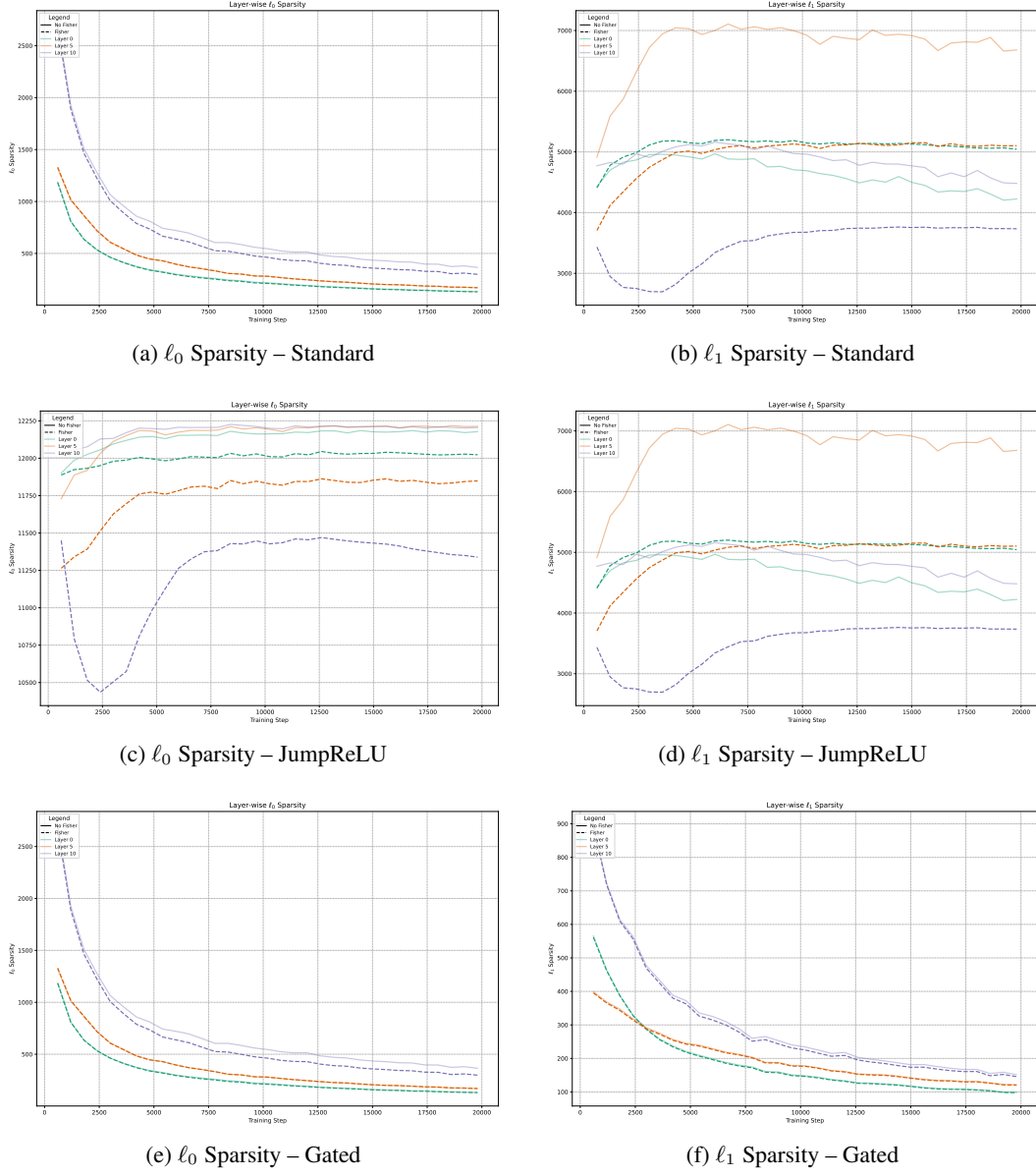


Figure 2: Sparsity metrics (ℓ_0 , ℓ_1) for MLP output activations across layers 0, 5, and 10.

Architecture	Component	ℓ_0	ℓ_1	ℓ_2 Ratio	MSE
Gated	MLP Out	-22.46	-2.55	0.00	-0.07
Gated	Residual	-30.99	-3.49	0.00	0.10
JumpReLU	MLP Out	-461.62	-518.66	0.00	0.13
JumpReLU	Residual	-2409.40	3.74	-0.01	0.00
Standard	MLP Out	-448.97	-501.19	0.00	0.11
Standard	Residual	-1605.63	-8.94	0.01	0.01

Table 2: Condensed comparison of average metric differences (Fisher - Non-Fisher) for SAEs trained with and without Fisher-based regularisation on Devign Negative values for ℓ_0 sparsity and MSE indicate desirable improvements (more compression, better reconstruction), while higher ℓ_1 suggests more structured sparsity. Highlighted cells mark notable improvements with Fisher regularisation.

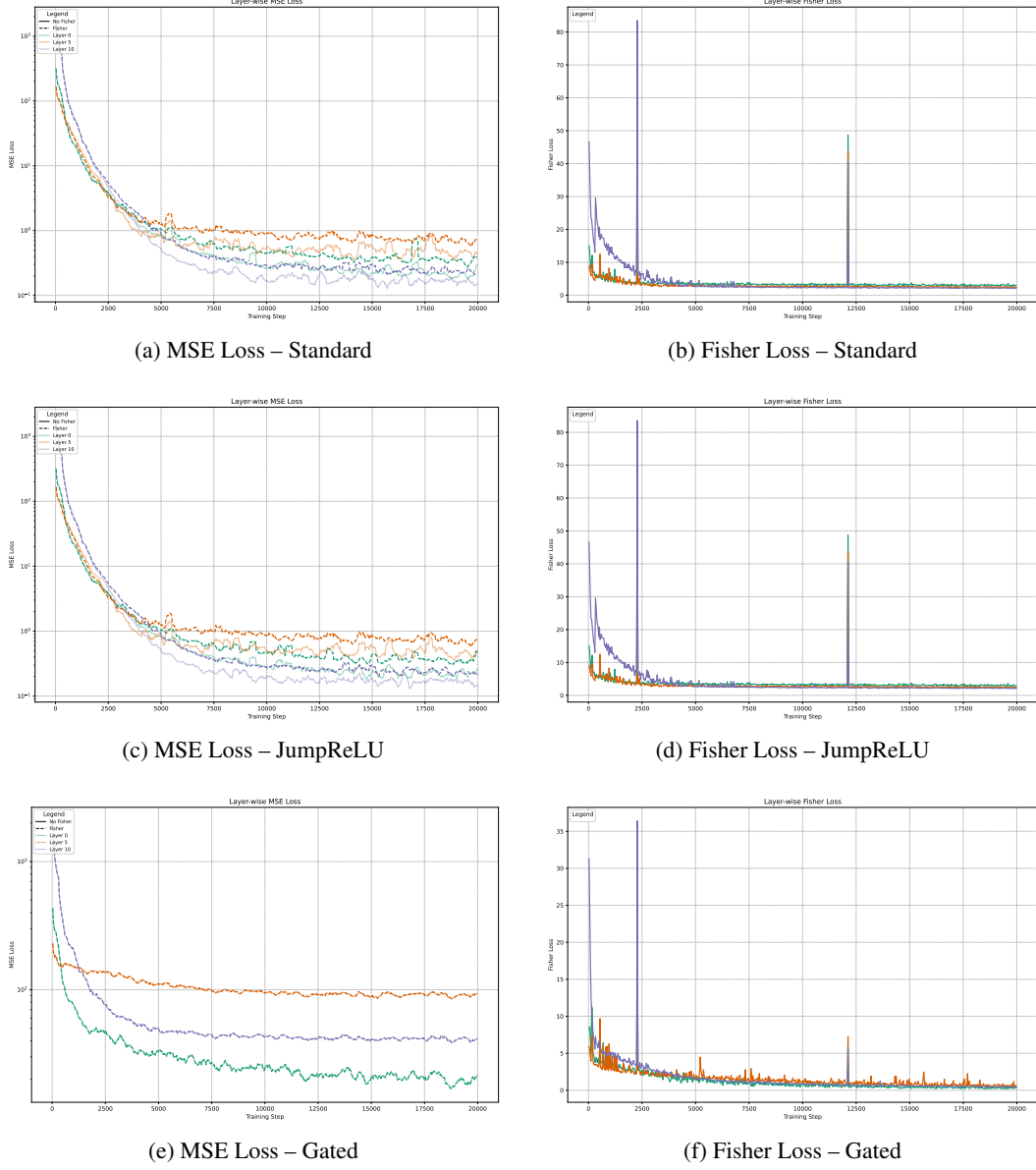
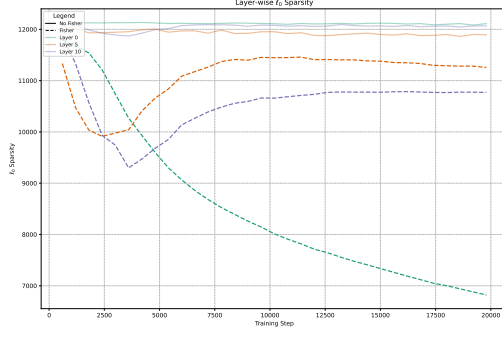


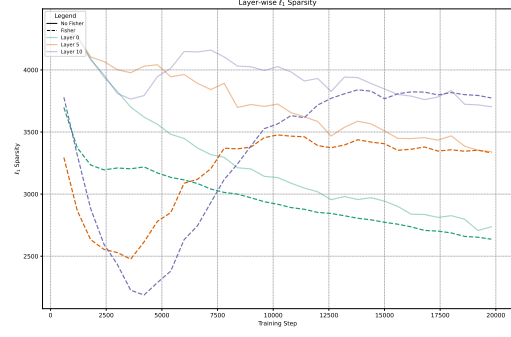
Figure 3: Reconstruction losses (MSE, Fisher) for MLP output activations across layers 0, 5, and 10.

Architecture	Component	ℓ_0	ℓ_1	ℓ_2 Ratio	MSE
Gated	MLP Out	-18.36	-2.55	0.00	0.04
Gated	Residual	-29.51	-3.21	0.00	-0.04
JumpReLU	MLP Out	-456.99	-986.07	0.00	0.26
JumpReLU	Residual	-2140.97	-44.62	-0.01	0.00
Standard	MLP Out	-444.44	-1007.11	0.00	0.16
Standard	Residual	-2128.21	-41.29	0.00	0.05

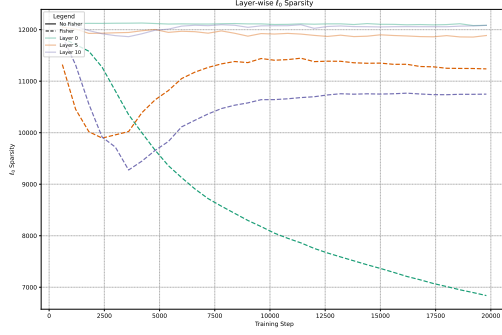
Table 3: Condensed comparison of average metric differences (Fisher - Non-Fisher) for SAEs trained with and without Fisher-based regularisation on the TheStack - Dedup Python. Negative values for ℓ_0 sparsity and MSE typically indicate improved compression and reconstruction, while subtle changes in ℓ_1 and ℓ_2 reflect structured sparsity and weight shrinkage. Highlighted cells mark substantial improvements.



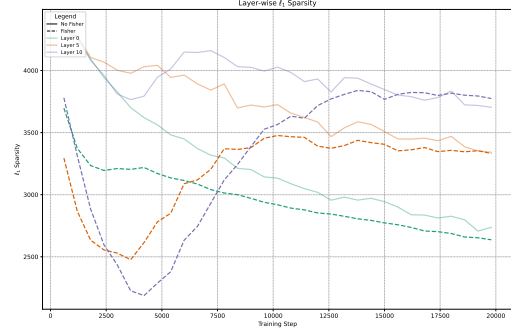
(a) ℓ_0 Sparsity – Standard Architecture



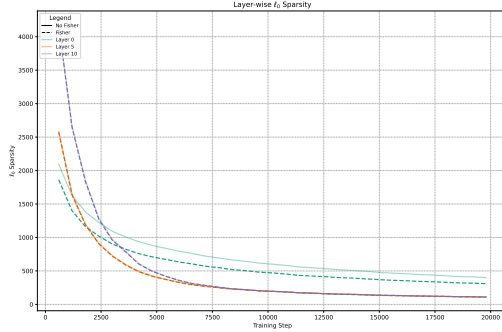
(b) ℓ_1 Sparsity – Standard Architecture



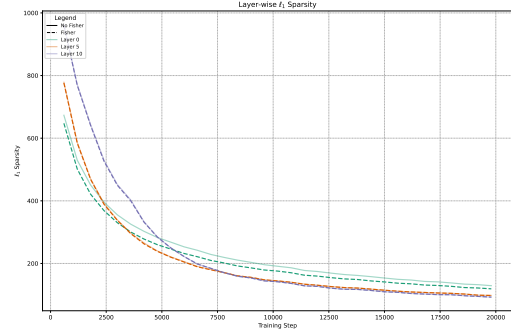
(c) ℓ_0 Sparsity – JumpReLU Architecture



(d) ℓ_1 Sparsity – JumpReLU Architecture



(e) ℓ_0 Sparsity – Gated Architecture

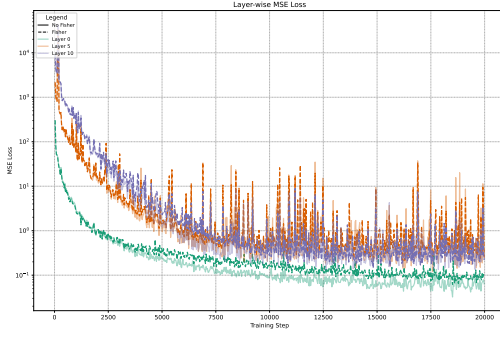


(f) ℓ_1 Sparsity – Gated Architecture

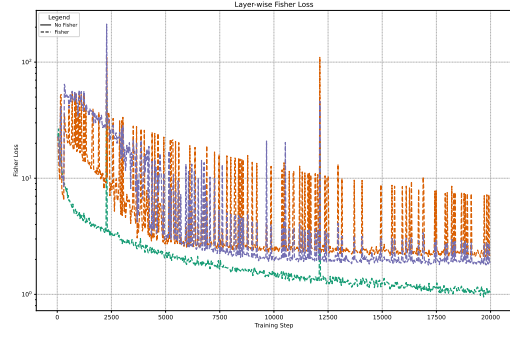
Figure 4: Sparsity metrics for residual stream activations across layers 0, 5, and 10.

Architecture	Component	ℓ_0	ℓ_1	ℓ_2 Ratio	MSE
Gated	MLP Out	-15.06	-2.03	0.00	0.18
Gated	Residual	-27.17	-2.83	0.00	-0.18
JumpReLU	MLP Out	-540.95	-1198.53	0.00	0.12
JumpReLU	Residual	-2002.30	-144.42	0.00	0.05
Standard	MLP Out	-518.12	-1173.56	0.00	0.17
Standard	Residual	-1984.46	-151.77	-0.01	0.05

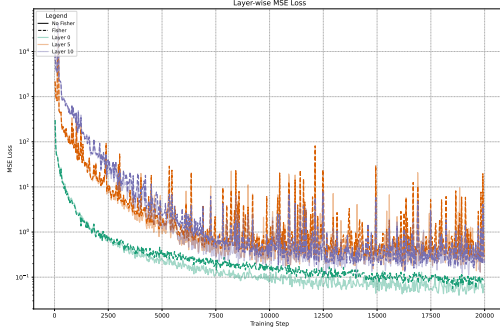
Table 4: Condensed comparison of average metric differences (Fisher - Non-Fisher) for SAEs trained with and without Fisher-based regularisation on the TheStack - Java. Negative values for ℓ_0 sparsity and MSE typically indicate improved compression and reconstruction, while subtle changes in ℓ_1 and ℓ_2 reflect structured sparsity and weight shrinkage. Highlighted cells mark substantial improvements.



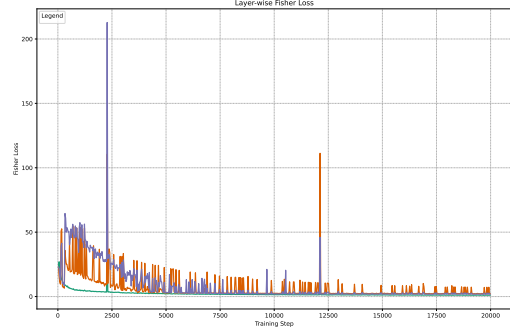
(a) MSE Loss – Standard Architecture



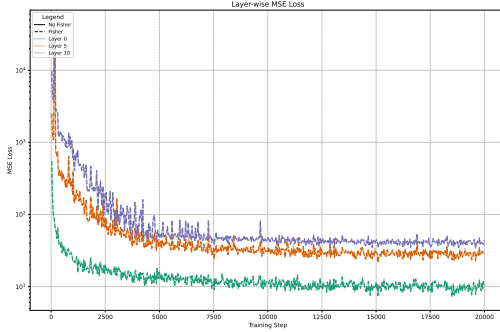
(b) Fisher Loss – Standard Architecture



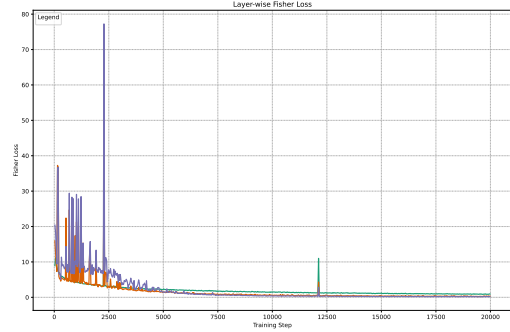
(c) MSE Loss – JumpReLU Architecture



(d) Fisher Loss – JumpReLU Architecture



(e) MSE Loss – Gated Architecture



(f) Fisher Loss – Gated Architecture

Figure 5: Reconstruction loss for residual stream activations across layers 0, 5, and 10.

Architecture	Component	ℓ_0	ℓ_1	ℓ_2 Ratio	MSE
Gated	MLP Out	-25.80	-2.50	0.00	-0.53
Gated	Residual Md	-27.50	-2.77	0.00	-0.07
JumpReLU	MLP Out	-602.81	-240.58	0.00	0.24
JumpReLU	Residual	-1902.54	-240.58	0.00	0.03
Standard	MLP Out	-579.10	-1531.92	0.00	0.36
Standard	Residual	-1883.72	-247.49	0.00	0.05

Table 5: Condensed comparison of average metric differences (Fisher - Non-Fisher) for SAEs trained with and without Fisher-based regularisation on the Tiny Stories. Negative values for ℓ_0 sparsity and MSE typically indicate improved compression and reconstruction, while subtle changes in ℓ_1 and ℓ_2 reflect structured sparsity and weight shrinkage. Highlighted cells mark substantial improvements.